

Week 1 – Introduction to Python and NumPy

DS3010: Introduction to Machine Learning Lab

Timing: 02:00 PM to 04:45 PM

Max Marks:10

Instructions

1. Submit one `.ipynb` file containing all answers. The name should be `[student_name]_[rollno]_[lab].ipynb`
2. Write the questions in separate text blocks before the answers.
3. Outputs for all sub-questions should be given and the code should be executable.
4. Write justifications for your choices where needed.
5. Ensure that all plots include clear labels and legends for better interpretation.
6. **Use of generative AI tools (such as ChatGPT, Gemini, etc.) is strictly prohibited. Any submission found to contain AI-generated or plagiarized content will receive a score of zero, and disciplinary action.**

1. Basics of Python

- (a). Take a list of numbers as user input. Print the sum of each number with its *previous* number. For the first element, there is no previous number, so handle this case appropriately.
- (b). Print the multiplication tables for all *odd* numbers between 11 and 25, up to 10 multiples each. Format the output neatly with a heading for each table.
- (c). Generate the first *20 terms* of the Fibonacci sequence. Then print only those terms that are *even*.
- (d). Check whether a given year is a leap year. A year is a leap year if it is divisible by 4, except for years that are divisible by 100 but not by 400. Print 'Yes, the given year is a leap year' or 'No, the given year is not a leap year'. Additionally, print the *next* leap year after the given year.
- (e). Take a string as user input and, using a single `for` loop, count the number of *digits* and *uppercase letters* it contains. If the string has no digits, also print 'No digits contained'.

2. Python Data Structures

- (a). Create an empty dictionary named `customer_transactions`.
- (b). Add records for 4 customers to the dictionary:

'Ankit'	: [1200, 5200, 75, 3400]
'Amit'	: [4500, 13000, 25, 900]
'Ravi'	: [200, 150, 10000, 60]
'Sneha'	: [7800, 450, 2300, 11500]
- (c). Display the transaction history for each account holder in a well-formatted way, clearly showing their name followed by their four transactions and the *average* transaction value (rounded to 2 decimal places).
- (d). Compute and print the *total transaction volume* for each account holder. Then print the customers in *descending* order of their total transaction value.
- (e). Identify and print the customer with the *highest total transaction value*.

- (f). A transaction is considered *suspicious* if it is greater than 10,000 or less than 100. Use a *dictionary comprehension* to create a new dictionary named `suspicious_customers` that maps each customer having at least one suspicious transaction to the *list of their suspicious transactions only*.

3. Basics of NumPy

- (a). Create a 2D NumPy array A of shape (5,4) with values from 1 to 20. Print the array, its *shape*, *size*, *number of dimensions*, and *data type*.
- (b). From the array A, extract and print:
- The *third column*.
 - A subarray containing the *last three rows* and the *first two columns*.
 - All elements of A that are *divisible by 3* (using boolean indexing).
- (c). Reshape A into a (4,5) array B. Create another array C of the same shape filled with the value 4. Perform element-wise *addition*, *subtraction*, *multiplication*, and *division*.
- (d). For array B, calculate and print the *row-wise* and *column-wise* sum, as well as the *mean*, *maximum*, and *minimum* along each axis.
- (e). Add the 1D array [5,10,15,20,25] to each row of B using *broadcasting*. Count how many elements in B are greater than 8 and print the count. Replace all elements in B that are less than 8 with 0, then print the final array.

4. Basics of Matplotlib

Given the following data for two cities' monthly average temperatures and rainfall:

```
months      = [Jan, Feb, Mar, Apr, May, Jun]
city1_temp  = [22, 25, 28, 30, 35, 40]
city2_temp  = [18, 20, 25, 27, 30, 32]
city1_rain  = [10, 8, 12, 20, 35, 80]
city2_rain  = [15, 12, 18, 25, 40, 95]
```

- (a). Plot a *line chart* showing both cities' temperatures across the months. Use different line styles and markers for each city. Add proper labels for the axes, a title, a legend, and a grid.
- (b). On a separate plot, draw a *grouped bar chart* comparing both cities' temperatures for *May and June only*, with appropriate labels and a legend.

Bonus Question (optional): On a separate plot, draw a *scatter plot* of temperature (x-axis) versus rainfall (y-axis) for both cities, using different colors for each city. Add a legend and briefly comment (in a text block) on any trend you observe.

All the best for your lab! Happy coding!